

Phase 1 Progress Report

Hyperbolic Deep Learning for Retinal Image Denoising

Author	Ayoub Hidara
Date	May 21, 2026
Phase	Phase 1 — Literature review and state of the art (2 weeks)
Status	In progress — major dataset deliverable complete, papers partially covered

Purpose of this document. A consolidated record of the Phase 1 work performed so far — the mathematical foundations studied, the papers reviewed, the dataset analysis, the design decisions taken and why, and what remains. Written to serve as a personal reference for the final internship report.

1. Project overview

The goal of the project is to develop a deep learning framework that leverages the **constant negative curvature of hyperbolic space** to improve the denoising of medical retinal imagery. Standard Euclidean convolutional networks struggle to represent the exponentially branching hierarchy of the retinal vascular tree without significant spatial distortion. By embedding image features in a space of constant negative curvature, the network gains an inductive bias suited to the fractal and hierarchical nature of retinal anatomy.

The project is structured into four phases: (1) literature review and state of the art, (2) architecture design and implementation, (3) training and optimization, (4) evaluation and final report. This document covers progress through Phase 1.

2. Phase 1 deliverables — status

Item	Status	Notes
Mathematical foundations of Poincaré ball + projectz models	Done	Poincaré ball + tangent space + exp/log + Möbius operations
Review existing denoising benchmarks	Pending	DnCNN, Noise2Noise/Noise2Void, Restormer/SwinIR — short
Review hyperbolic convolutional networks	Partial	Nickel & Kiela 2017 + Ganea et al. 2018 completed. HNN+ =
Analyze noise characteristics of target dataset	Done	Empirical Poisson-Gaussian fit on RFMiD attempted, found u
(Stretch) Baseline Euclidean U-Net standard	Next	Strictly a Phase 2 task, but starting it now de-risks downstre

3. Mathematical foundations

3.1 Why hyperbolic space for hierarchical data

In a tree with branching factor b , the number of nodes at distance d from the root grows like b^d — exponential in d . In Euclidean space $\mathbb{R}^n$, the volume of a ball of radius r grows only polynomially (like r^n). As a tree gets deeper, distinct leaves get squashed together and the embedding distorts the tree structure.

Hyperbolic space, by contrast, has the property that **volume grows exponentially with radius**. This matches the structure of hierarchical data exactly. It is also the right inductive bias for the retinal vascular tree, which branches recursively from the optic disc through arterioles, capillaries, and finally microvessels — with an effective branching factor and depth that produce exponentially many fine terminal structures.

3.2 The Poincaré ball model

The Poincaré ball $\mathbb{B}^n$ is the open unit ball in $\mathbb{R}^n$ ($\{x : \|x\| < 1\}$), equipped with a non-Euclidean way of measuring distance. The Poincaré distance between two points u, v inside the ball is:

$$d(u, v) = \operatorname{arccosh} \left(1 + 2 \cdot \|u - v\|^2 / ((1 - \|u\|^2) \cdot (1 - \|v\|^2)) \right)$$

Key intuition: the denominator $(1 - \|u\|^2)(1 - \|v\|^2)$ approaches zero as either point approaches the boundary of the ball. So the hyperbolic distance blows up near the boundary, even though the Euclidean distance is bounded. Equivalently: **the boundary of the ball is infinitely far away** in hyperbolic terms. This is what gives us the room to embed arbitrarily deep hierarchies.

Visual analogy: Escher's *Circle Limit* prints. Angels and devils get smaller as they approach the edge, but in the hyperbolic metric they are all the same size.

3.3 Tangent spaces and the exp / log maps

The Poincaré ball is **not a vector space** — you cannot freely add or scale points and remain inside the ball. To do linear algebra on hyperbolic points, we move to the **tangent space** at a reference point (typically the origin).

Informally, the tangent space at point x — written $T_x M$ — is the flat hyperplane that best approximates the manifold M at x . The crucial property is that the tangent space *is* a regular vector space ($\mathbb{R}^n$), even when the manifold itself is not. Inside it we can add, scale, apply ReLU, do anything Euclidean.

Two functions move between the manifold and the tangent space at the origin (Poincaré ball, curvature $c = 1$):

$$\begin{aligned} \exp_0(v) &= \tanh(\|v\|) \cdot v / \|v\| \quad (\text{tangent vector} \rightarrow \text{ball point}) \\ \log_0(y) &= \operatorname{arctanh}(\|y\|) \cdot y / \|y\| \quad (\text{ball point} \rightarrow \text{tangent vector}) \end{aligned}$$

Two sanity properties: (a) for small $\|v\|$, $\tanh(\|v\|) \approx \|v\|$, so $\exp_0$ is approximately the identity near the origin — the formal sense in which hyperbolic space looks locally Euclidean; (b) $\operatorname{arctanh}$ blows up as $\|y\| \rightarrow 1$, mirroring the geometric fact that the boundary is infinitely far. The $\operatorname{arctanh}$ blow-up is also the source of all the **numerical stability** headaches in hyperbolic neural networks — every implementation clamps $\|y\|$ to something like $1 - 10^{-5}$ before calling $\operatorname{arctanh}$.

3.4 Why vanilla layers fail on the Poincaré ball

A standard feedforward layer does three things: matrix-vector product ($y = W \cdot x$), vector addition ($y' = y + b$), and a pointwise nonlinearity ($z = \text{ReLU}(y')$). In $\mathbb{R}^n$ all three are free; on the Poincaré ball, all three break:

- **Closure fails.** If $u = (0.6, 0)$ is in $\mathbb{B}^2$, then $2 \cdot u = (1.2, 0)$ is outside the ball. Sum, scalar multiple, matrix multiply — all can push the result outside.
- **Geometric meaningless.** Even when the output stays inside, coordinate-wise operations like ReLU don't correspond to any natural notion of "translation" or "truncation" on a curved space.

What we need: operations that stay inside the manifold *and* match the geometric notion of "add" or "scale" in the curved space. That's the entire job of **Möbius algebra**.

3.5 Möbius operations

Möbius scalar multiplication (curvature $c = 1$):

$$r \otimes x = \tanh(r \cdot \operatorname{arctanh}(\|x\|)) \cdot x / \|x\|$$

Worked example: $2 \otimes (0.6, 0)$. $\operatorname{arctanh}(0.6) \approx 0.6931$, $2 \cdot 0.6931 = 1.3863$, $\tanh(1.3863) \approx 0.8825$. So $2 \otimes (0.6, 0) = (0.8825, 0)$. Compare to ordinary Euclidean doubling, which would give $(1.2, 0)$ — outside the ball. Möbius doubling stayed inside, at 0.8825. The hyperbolic notion of "twice as far" reaches much further along the manifold than the Euclidean coordinates suggest, because near the boundary the manifold is exponentially stretched relative to the coordinates.

Möbius addition:

$$u \oplus v = ((1 + 2\langle u, v \rangle + \|v\|^2) \cdot u + (1 - \|u\|^2) \cdot v) / (1 + 2\langle u, v \rangle + \|u\|^2 \cdot \|v\|^2)$$

Worked example: $(0.5, 0) \oplus (0, 0.5) \approx (0.588, 0.353)$, but $(0, 0.5) \oplus (0.5, 0) \approx (0.353, 0.588)$. The two sums are reflections of each other across the diagonal — **Möbius addition is non-commutative**, and also non-associative.

The intuition for non-commutativity: in a curved space, walking from origin to point A actually *rotates your frame of reference* by an amount that depends on the curvature you traversed (a phenomenon called **holonomy**). So "walk to A, then walk by B" and "walk to B, then walk by A" end at points that differ by a rotation — algebraically, $u \oplus v = \operatorname{gyr}[u, v](v \oplus u)$. This is the origin of the term *gyrovector space* (Ungar's framework).

3.6 The tangent-space sandwich pattern

Combining $\exp / \log$ with the Möbius operations gives a universal recipe: lift any Euclidean operation f into the manifold via the sandwich

$$f^\otimes(x) = \exp_0(f(\log_0(x)))$$

This gives almost the whole library of hyperbolic neural network components:

- **Hyperbolic linear layer** (Möbius matrix-vector product): $M \otimes x = \exp_0(M \cdot \log_0(x))$.
- **Hyperbolic activation** (Möbius ReLU, tanh, GELU, ...): $f^\otimes(x) = \exp_0(f(\log_0(x)))$.
- **Hyperbolic scalar multiplication**: identical sandwich.

Practical implication for the implementation: most of the computation in a hyperbolic network actually happens *in tangent coordinates* (where everything is plain $\mathbb{R}^n$ and behaves normally), with exp / log brackets at the boundary. Möbius addition is only used at explicit "bias add" or "residual" moments, and the order matters there.

4. Papers reviewed

4.1 Nickel & Kiela (2017) — Poincaré Embeddings for Learning Hierarchical Representations

Core idea. Embed objects (WordNet nouns, social-network nodes) as points in the Poincaré ball, optimized to keep semantically related pairs close (in hyperbolic distance) and unrelated pairs far. Optimization is done with Riemannian SGD, which rescales the Euclidean gradient by the inverse metric tensor of the ball (effectively by $(1 - \|x\|^2)^2 / 4$) and projects back into the ball after each step.

Headline result. A 5-dimensional Poincaré embedding beat a 200-dimensional Euclidean embedding on WordNet hierarchy reconstruction. Forty times fewer dimensions, better performance. This was the result that established hyperbolic geometry as a first-class tool for representing hierarchies.

What to retain. (1) The exponential-volume argument — why hyperbolic matches hierarchies. (2) The Poincaré distance formula and the "boundary is infinitely far" intuition. (3) The need for Riemannian optimization when parameters live on a manifold.

4.2 Ganea, Bécigneul & Hofmann (2018) — Hyperbolic Neural Networks

Core idea. Nickel & Kiela embedded points but did not have any neural network *layers* on hyperbolic vectors. Ganea provides the construction manual: how to define linear maps, biases, activations, classifiers, and RNNs on points in $\mathbb{B}^n$ such that everything stays differentiable and within the manifold.

Key contributions. (a) The tangent-space sandwich pattern ($\exp \circ \text{Euclidean op} \circ \log$) for linear maps and activations. (b) Möbius addition for biases and residuals, with care for non-commutativity. (c) Hyperbolic multinomial logistic regression ("hyperbolic softmax") by generalizing the notion of geodesic hyperplanes. (d) Hyperbolic GRU by Möbius-ifying every component of the standard GRU update equations.

Empirical result. Hyperbolic versions match or exceed Euclidean ones on textual entailment and link prediction, at substantially lower dimension.

Why it matters for this project. Phase 2's planned **exponential map projection layer** and **hyperbolic bottleneck** are direct applications of Ganea's construction. The architecture sketch in Section 5 below shows where each piece slots in.

5. Architecture sketch

The Phase 2 architecture keeps a standard U-Net skeleton (encoder-bottleneck-decoder with skip connections) and inserts hyperbolic operations only at the bottleneck. The rationale: pixel-level features are not hierarchical objects — patches of pixels are. By the time the encoder has reduced spatial resolution and built abstract features, the representation is hierarchical in nature, and hyperbolic geometry pays off.

5.1 Standard Euclidean U-Net (baseline)

Forward pass at the bottleneck (pseudocode):

```
x : (B, C, H, W) – feature map in  $\mathbb{R}^n$ 
x = Conv(x)
x = BatchNorm(x)
x = ReLU(x)
x = Conv(x)
return x – still in  $\mathbb{R}^n$ 
```

5.2 Hyperbolic-augmented U-Net

Bottleneck replaced by the sandwich pattern:

```
x : (B, C, H, W) – feature vectors in  $\mathbb{R}^n$ 
x_h = exp_0(x) # lift to  $\mathbb{H}^n$ 
y_h = möbius_linear(M, x_h) # = exp_0( M · log_0(x_h) )
y_h = möbius_add(y_h, b) #  $y_h \oplus b$  (non-commutative)
y_h = möbius_activation(y_h) # = exp_0( ReLU( log_0(y_h) ) )
x = log_0(y_h) # drop back to  $\mathbb{R}^n$ 
return x – for the decoder
```

Skip connections are unchanged. They carry raw Euclidean features around the hyperbolic bottleneck. This is critical for denoising: the high-frequency pixel detail needed to reconstruct sharp edges should not be forced through the hyperbolic layer (which would blur it). The hyperbolic path handles *structure*; the skip path handles *texture*; they recombine in the decoder.

Stretch goal (later phase). Replace each Euclidean convolution with a *hyperbolic* convolution (Bdeir et al. 2024, HCNN). The whole network then lives in $\mathbb{B}^n$ end-to-end with a single exp at input and single log at output. More principled, more expensive, less stable to train.

6. Dataset analysis — RFMiD

6.1 Dataset overview

RFMiD (Retinal Fundus Multi-disease Image Dataset), distributed on Kaggle by andrewmvd. Approximately 3,200 colour fundus photographs labelled across 45+ retinal disease categories. The dataset comes with predefined Training_Set / Evaluation_Set / Test_Set splits.

Critical observation: RFMiD is a **classification** dataset, not a denoising dataset. There are no paired (clean, noisy) images — each retina was photographed once. The labels are diagnoses, not denoised versions of anything.

6.2 Strategy decision: synthetic noise on clinical images

Three realistic options were considered:

- **Option A (chosen):** treat RFMiD images as pseudo-clean ground truth, corrupt them with a simulated noise model, and train supervised on the synthetic (noisy, clean) pairs. This is what most retinal-denoising papers do in the absence of true paired data.
- **Option B:** self-supervised (Noise2Void / Noise2Self) — trains on single noisy images by masking and predicting. No clean targets needed but assumes pixel-independent noise. Kept as a possible Phase 4 ablation.
- **Option C:** use the RIADD Image Quality Assessment subset to denoise low-quality images using good-quality references. Not really denoising since you don't have the same retina in both states — closer to quality enhancement.

6.3 Empirical noise estimation attempt

An attempt was made to fit a Poisson-Gaussian noise model directly to RFMiD images. The methodology:

1. Inspect the dataset (resolutions, channels, intensity histograms). Confirmed the standard RFMiD geometry: circular retinal disc inside a square frame, large near-zero spike in all three colour channels, with the green channel having the widest mid-range distribution.
2. Use the **green channel** for noise analysis — highest vessel contrast, widest dynamic range, standard convention in retinal imaging.
3. Slide a non-overlapping 8×8 window over the image, keep patches whose mean is in [15, 240] (excluding pure black border and near-saturated pixels) and whose variance is below 30 (excluding vessel-edge patches dominated by signal gradients).
4. Fit the Poisson-Gaussian model $\text{var}(y) = \alpha \cdot \mu + \sigma^2$ by ordinary least squares over all (mean, variance) pairs collected from ~50 sampled images.

Visual sanity check at step 3 was excellent: the accepted-patch mask correctly covered the smooth retinal background while excluding the black border and the vessel tree.

6.4 Why the empirical fit failed

The fit returned $\alpha \approx 0.020$ and $\sigma \approx 1.87$, which produces noise so small it is essentially **invisible** to the human eye. The validation step (synthetic noisy minus clean) produced

an almost completely black difference map, confirming the model adds essentially nothing.

The cause is the nature of RFMiD itself. The dataset consists of **post-processed clinical images**: each image has been through white balance, gamma correction, sharpening, and JPEG compression. The original sensor noise has been distorted beyond recognition. What remains is a non-Gaussian, spatially correlated residual dominated by JPEG 8×8 block artifacts and slow brightness gradients — none of which a simple Poisson-Gaussian model captures.

Note the unfortunate coincidence: our 8×8 analysis patches align exactly with JPEG's 8×8 DCT blocks, so the patch-variance estimates are largely measuring JPEG quantization patterns rather than true sensor noise.

6.5 Pivot to literature-based noise levels

Following standard practice in the retinal-denoising literature, three Poisson-Gaussian noise levels were adopted (representative of light, medium, and heavy degradation in 8-bit fundus images):

Level	α (shot)	σ (read)	Noise std at $\mu = 128$
light	0.5	3.0	≈ 8
medium	1.0	5.0	≈ 12
heavy	2.0	8.0	≈ 18

Training strategy: for each training image, sample one of the three noise levels uniformly at random. The U-Net thus learns to denoise across a range of noise severities, yielding a *blind denoiser*. At evaluation time the model is tested at each level separately to give a clean robustness story (three PSNR / SSIM numbers per model).

6.6 Deliverables

A Jupyter notebook (**rfmid_noise_analysis.ipynb**, 29 cells) executes the full analysis on Kaggle: dataset inspection, empirical fit attempt, visualization of the failure, definition of the three adopted noise levels, side-by-side visualization of clean vs the three noisy versions, and a **noise_config.json** output that the U-Net dataloader will read. The empirical attempt is preserved in the notebook because the discovery *why* it failed is itself a legitimate methodological finding worth reporting.

7. Decisions log

The following decisions have been made during Phase 1. Each is recorded with its rationale, so that the reasoning can be cited in the final report or revisited if circumstances change.

Decision	Choice	Rationale
Task framing	Stays as denoising (not switched to classification)	The guidelines specify denoising. Supervisor's comment: "The project is about denoising, not classification."
Backbone architecture	U-Net (segmentation-oriented)	Explicitly required for Phase 2 of the guidelines. The encoder-decoder + skip connections fit the hyperbolic projection layer.
Hyperbolic model	Lorentz (hyperboloid)	Specified by the guidelines for the projection layer. Numerically more stable than Poincaré disk.
Dataset	RFMiD (~3,200 fundus images)	Already available, classification labels not needed for our purposes; training on denoising is sufficient.
Noise model	Poisson-Gaussian at the pixel level	Empirical test on RFMiD showed it was unreliable (JPEG-dominated). Three levels of noise (0.1, 0.2, 0.3) were used for training.
Training compute	Kaggle (free GPU)	No local GPU available. Kaggle has the dataset pre-attached, free T4 GPU (16GB).
Phase 1 vs Phase 2 boundary	Stand up Euclidean baseline first	Strictly a Phase 2 task, but doing it now de-risks dataloader / training pipeline.

8. Remaining work for Phase 1

Critical-path items (block Phase 2):

- **Lorentz / hyperboloid model.** Phase 2 specifies it for the projection layer. Need to cover: bilinear form, hyperboloid constraint, exp/log in Lorentz coordinates, why Lorentz is numerically more stable than the Poincaré ball, isometry between the two.
- **Numerical stability deep dive.** The arctanh blow-up, projection clamping, why mixed-precision is dangerous for hyperbolic operations.
- **Riemannian optimization (RSGD / RAdam).** Why ordinary SGD / Adam does not work on manifold parameters; how the geoopt PyTorch library provides everything for free.

Important but not blocking:

- Shimizu et al. *Hyperbolic Neural Networks++* (2021) — refinements to Ganea's HNN.
- Bdeir et al. *Fully Hyperbolic CNN* (HCNN, 2024) — hyperbolic convolutions, Lorentz batch normalization. Read before attempting the fully-hyperbolic stretch goal.
- Denoising baselines crash course: DnCNN, Noise2Noise / Noise2Void, Restormer / SwinIR.
- Retinal denoising metrics (PSNR, SSIM, vessel-continuity / topological fidelity) plus one recent MICCAI or CVPR paper applying hyperbolic geometry to medical imaging.

9. Immediate next steps

1. Verify the updated noise notebook runs cleanly on Kaggle and the three-level visualization renders correctly. Save noise_config.json.
2. Share the noise-level visualization with the supervisor; confirm she is comfortable with the three-level training strategy (or accept a different intensity range).
3. Build the baseline Euclidean U-Net: PyTorch Dataset reading RFMiD + on-the-fly noise injection, plain U-Net architecture, training loop with MSE + SSIM, evaluation script producing PSNR / SSIM per noise level and a few before/after PNG examples.
4. Train on Kaggle (free GPU); produce reference numbers for the Euclidean baseline.
5. In parallel, complete the remaining Phase 1 papers (Lorentz model, HNN++, HCNN).
6. Phase 2 proper begins once the Euclidean baseline trains successfully and the Lorentz math has been internalized.

10. Artefacts produced

Files generated and saved to the project folder:

- **rfmid_noise_analysis.py** — 374-line Python script with `###` cell markers; the source of truth for the notebook.
- **rfmid_noise_analysis.ipynb** — 29-cell Jupyter notebook, Kaggle-ready. Documents the empirical noise estimation, its failure, the pivot, and the adopted three-level noise model.

- **noise_config.json** (produced by running the notebook) — the configuration file consumed by the U-Net dataloader.
- **phase1_report.pdf** — this document.